Backtesting Tool Advanced Proposal

Securities Firm Grade Assessment
& Implementation Blueprint

BRUTAL HONEST GAP ANALYSIS

Version 2.0 | 2026.04

From Toy Project to Institutional-Grade Platform

Table of Contents

Part	Section
Part 1	Brutal Honest Assessment - 현재 툴의 냉정한 평가
Part 2	9 Critical Gaps - 증권사가 절대 안 사는 9가지 이유
Part 3	Must-Have: Real-time Data Pipeline
Part 4	Must-Have: Compliance & Risk Controls
Part 5	Must-Have: Stress Testing & Scenario Analysis
Part 6	Must-Have: Multi-tenancy & Audit Trail
Part 7	Should-Have: Transaction Cost Analysis (TCA)
Part 8	Should-Have: Performance Attribution
Part 9	Architecture Blueprint
Part 10	Business Model & Pricing
Part 11	Implementation Roadmap

Part 1: Brutal Honest Assessment

현재 상태로 증권사에 팔 수 있는가? NO.

솔직하게 말하겠습니다. 지금 우리 툴은 **개인 투자자용 교육 도구**로는 훌륭합니다. 하지만 증권사에 납품하려면 **완전히 다른 물건**이 필요합니다. 맛있는 집밥과 레스토랑 요리의 차이입니다.

현재 점수표

영역	현재 점수	증권사 기준	Gap
데이터 신뢰성	F	A+	Yahoo Finance는 지연·누락이 잦고, SLA가 없음. 증권사는 Bloomberg/Reuters급 필요
실시간 처리	F	A	현재 요청-응답 구조. 증권사는 실시간 스트리밍 필수
리스크 컨트롤	C	A+	Sharpe/MDD는 있지만, 실시간 한도관리·자동 손절·포지션 한도가 없음
컴플라이언스	F	A+	감사 추적(Audit Trail) 없음. 금감원 보고 기능 없음. 이것만으로 탈락
인증/권한	F	A	로그인 없음. 증권사는 역할별 접근 제어(RBAC) 필수
확장성	D	A	단일 서버. 동시 사용자 100명만 되어도 죽음
전략 깊이	B	A	기본 전략은 있으나, 커스텀 전략 코딩·백테스트 엔진이 없음
시각화	B+	A	Recharts 기반으로 괜찮지만, 실시간 대시보드·드릴다운이 없음
API 연동		A	

D	OMS/EMS 연동 없음. 주문 실행 시스템과 붙을 수 없음

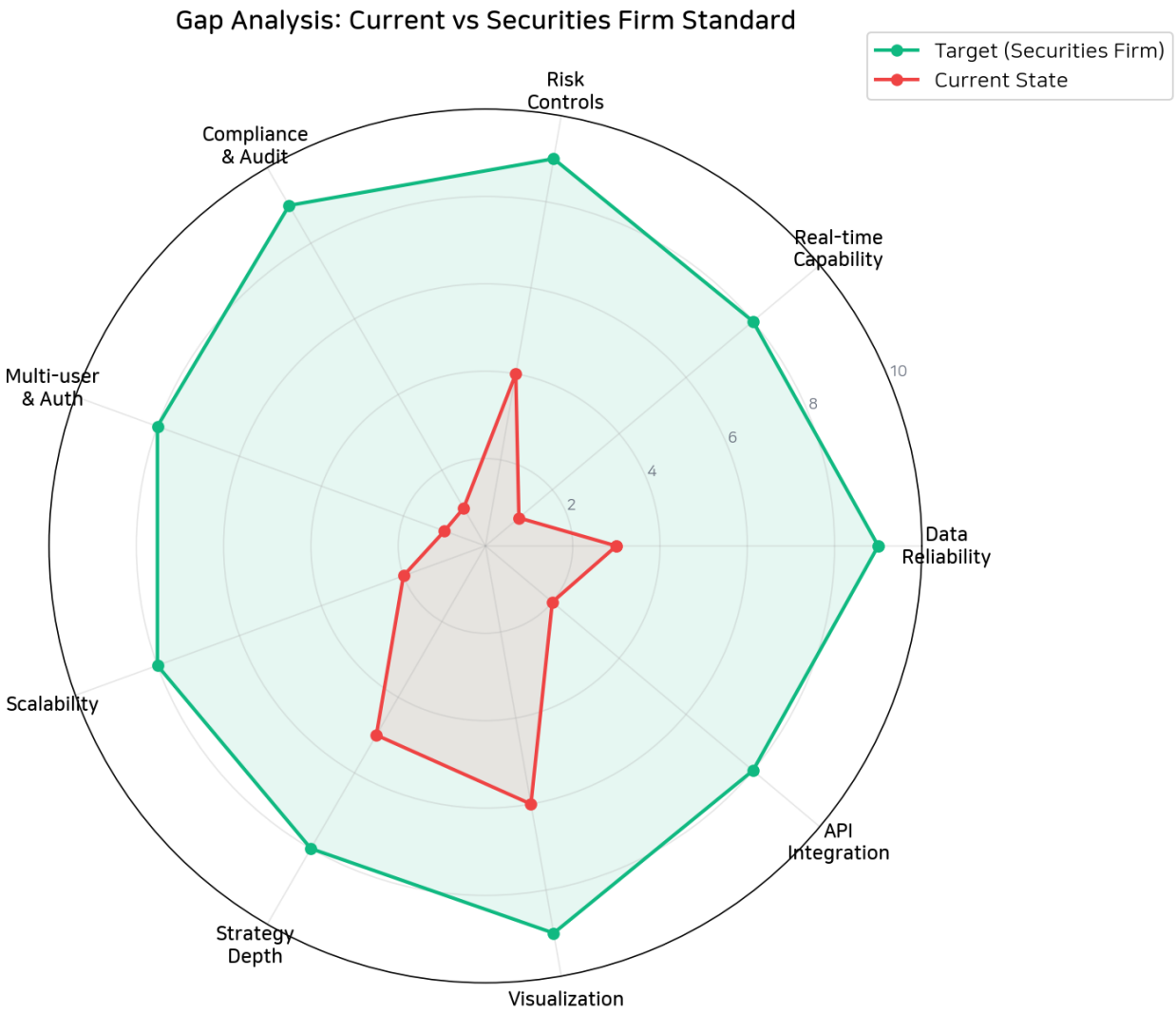


Figure 1. Gap Analysis Radar - 빨간색(현재) vs 초록색(증권사 기준). 격차가 클수록 문제.

핵심 문제: "장난감"과 "무기"의 차이

현재 툴은 **"과거에 이렇게 했으면 이만큼 벌었을 텐데"**를 보여줍니다. 증권사가 필요한 건 **"지금 이 포지션을 잡으면 얼마의 리스크가 있고, 컴플라이언스를 통과하는가"**입니다. 시제가 다릅니다. 과거형 vs 현재형+미래형.

Part 2: 증권사가 절대 안 사는 9가지 이유

증권사 IT 담당자가 우리 툴을 보면 이런 반응을 할 겁니다:

증권사 CTO의 예상 피드백

"시각화 예쁘고 컨셉 좋은데, 이거 프로덕션에 못 넣습니다."

"Yahoo Finance요? 장중에 데이터 끊기면 누가 책임져요?"

"감사 로그 없으면 금감원 검사 때 우리가 과태료 맞습니다."

"동시 접속 몇 명까지 됩니까? 우리 트레이더만 200명입니다."

9가지 Gap 상세

Gap 1: 데이터 소스가 장난감 수준

현재: Yahoo Finance (무료, SLA 없음, 지연 15분+, 언제든 차단 가능)

필요: Bloomberg Terminal API ($24,000/년), Refinitiv Eikon, 또는 KRX 직접 피드

왜 문제인가: 트레이더가 오전 9시 30분에 포트폴리오 가치를 조회하는데, Yahoo Finance가 15분 전 데이터를 보여주면? 그 15분 동안 주가가 5% 빠졌을 수 있습니다. **증권사에서 "대략적인 데이터"는 존재하지 않습니다.**

비유

수술실에서 환자 모니터가 15분 전 심박수를 보여주는 것과 같습니다. "대략 괜찮겠지"는 통하지 않습니다.

Gap 2: 실시간 처리 능력 제로

현재: 사용자가 버튼을 눌러야 데이터를 가져옴 (Request-Response)

필요: WebSocket으로 실시간 가격 스트리밍, 포지션 자동 업데이트, 리스크 한도 실시간 모니터링

왜 문제인가: 리스크 관리 부서는 VaR가 한도를 넘으면 **즉시** 알아야 합니다. "새로고침 눌러보세요"는 답이 아닙니다.

Gap 3: 리스크 한도 관리 없음

현재: Sharpe, MDD를 "보여주기만" 함

필요:

- **Pre-trade check:** 주문 전에 "이 주문을 체결하면 VaR 한도를 초과하는가?" 자동 검사
- **Real-time limit monitoring:** 포지션 한도, 섹터 한도, 국가별 한도 실시간 감시
- **Automatic stop-loss:** MDD가 설정 수준 초과 시 자동 포지션 축소
- **Margin call alert:** 담보 비율 하락 시 즉시 알림

Gap 4: 컴플라이언스 & 감사 추적 전무

현재: 누가 언제 뭘 했는지 기록 없음

필요:

- **Audit Trail:** 모든 조회, 주문, 설정 변경을 기록. 변경 불가능한 로그
- **금감원 보고:** 일일 VaR 보고서, 대량 거래 보고, 내부자 거래 감시
- **MiFID II / K-IFRS 준수:** 거래 기록 보관 (최소 5년)

> **이것만으로 탈락**
>
> 금융감독원은 모든 금융기관에 **거래 추적 시스템**을 의무화하고 있습니다. Audit Trail이 없는 시스템은 **아무리 기능이 좋아도 도입 불가**입니다. 법적 요건입니다.

Gap 5: 인증/권한 체계 없음

현재: 누구나 모든 기능에 접근 가능

필요:

- **RBAC (Role-Based Access Control):** 트레이더는 자기 포트폴리오만, 리스크팀은 전체 조회, 경영진은 대시보드만
- **SSO (Single Sign-On):** 증권사 기존 인증 시스템(AD/LDAP)과 연동
- **2FA:** 고위험 작업(대량 주문, 설정 변경)에 이중 인증

Gap 6: 확장성 & 고가용성

현재: 단일 Spring Boot 서버. DB도 없음

필요:

- 동시 사용자 500명+ 지원
- 99.9% 이상 가동률 (연간 다운타임 8.7시간 이내)
- 장애 시 자동 복구 (Active-Standby 또는 Active-Active)

Gap 7: Stress Testing

현재: 없음

필요: "2008년 금융위기가 다시 오면 우리 포트폴리오는 어떻게 되는가?" 이 질문에 즉시 답할 수 있어야 합니다.

Gap 8: Transaction Cost Analysis (TCA)

현재: 단순 수수료 추정

필요: 실제 체결 가격 vs 의사결정 시점 가격 차이 분석. "슬리피지가 얼마인가?" "시장 충격이 얼마인가?"

Gap 9: OMS/EMS 연동

현재: 분석만 가능. 실제 주문 실행 불가

필요: 분석 결과를 바로 주문으로 전환. Order Management System과 FIX 프로토콜 연동.

Part 3: Must-Have — Real-time Data Pipeline

왜 필수인가?

증권사의 모든 시스템은 **실시간 데이터** 위에 돌아갑니다. 15분 지연 데이터로 리스크를 관리하는 건, 백미러만 보고 운전하는 것과 같습니다.

구현 방법 (비전공자용)

실시간 데이터 파이프라인은 이렇게 생겼습니다:

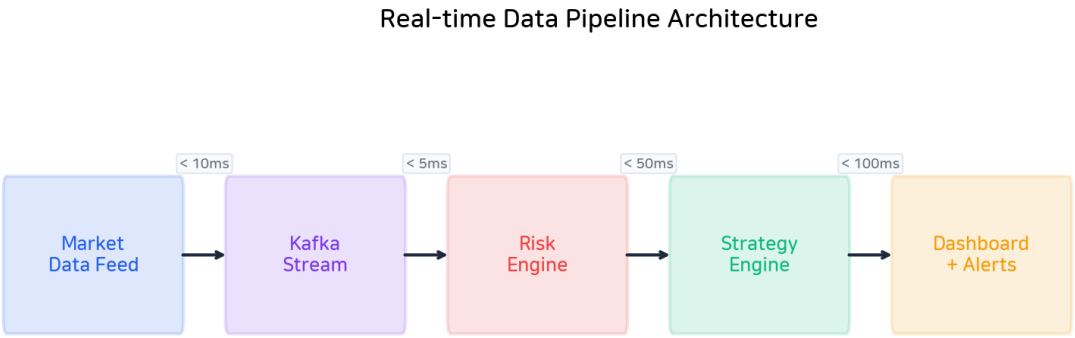


Figure 2. Real-time Data Pipeline Architecture

각 단계 설명

단계	역할	비유
Market Data Feed	거래소에서 실시간 가격 데이터를 받아옴	뉴스 특파원이 현장에서 생방송으로 보도하는 것
Kafka Stream	초당 수만 건의 데이터를 순서대로 정리하고 분배	공항 수하물 컨베이어 벨트. 짐이 아무리 많아도 순서대로 처리
Risk Engine	새 가격이 올 때마다 VaR, 포지션 한도를 재계산	자동차 계기판. 속도가 변할 때마다 즉시 표시
Strategy Engine	전략 조건 충족 시 매매 신호 생성	

		교통 신호등. 조건에 따라 초록불/빨간불 자동 전환
Dashboard + Alerts	결과를 WebSocket으로 실시간 화면에 표시 + 알림	주식 시세 전광판 + 긴급 문자

기술 스택

컴포넌트	기술	왜 이 기술인가
데이터 피드	KRX FAST Protocol / Bloomberg B-PIPE	거래소 공식 피드. 지연 없음. SLA 보장
메시지 큐	Apache Kafka	초당 100만+ 메시지 처리. 금융권 표준
시계열 DB	TimescaleDB 또는 QuestDB	시간순 데이터에 최적화. 일반 DB보다 100배 빠른 시계열 쿼리
실시간 전송	WebSocket + Server-Sent Events	서버→클라이언트 실시간 푸시. 새로고침 불필요
캐시	Redis	최신 가격을 메모리에 보관. 1ms 이내 응답

비용 현실

Bloomberg Terminal 라이선스: 연 $24,000/사용자. Refinitiv: 연 $12,000~. KRX 직접 피드: 상대적으로 저렴하지만 인프라 구축 필요. **데이터 비용만으로 수천만원/년.** 이것이 증권사 툴 가격이 비싼 이유입니다.

Part 4: Must-Have — Compliance & Risk Controls

왜 필수인가?

2024년 한국 금융감독원은 내부통제 위반으로 총 **과태료 387억원**을 부과했습니다. 컴플라이언스는 "있으면 좋은 것"이 아니라 **"없으면 영업정지"**인 기능입니다.

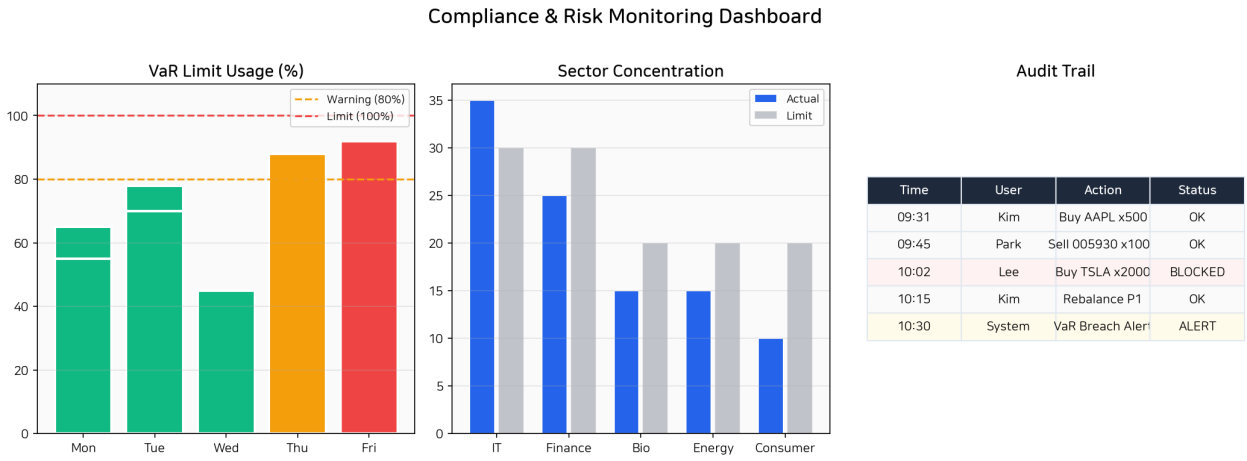


Figure 3. 컴플라이언스 대시보드 목업 - VaR 한도, 섹터 집중도, 감사 로그

구현해야 할 것들

1. Pre-trade Risk Check (사전 리스크 검사)

이게 뭔가요? 트레이더가 "삼성전자 1만주 매수" 버튼을 누르면, 실제 주문이 나가기 **전에** 시스템이 자동으로 검사합니다:

- "이 주문을 체결하면 VaR 한도를 초과하는가?"
- "이 종목 보유 한도를 넘는가?"
- "이 섹터 비중이 내부 규정을 초과하는가?"
- "이 주문이 시장 조작으로 의심될 수 있는가?"

비유: 놀이공원에서 키 제한처럼, 조건 미충족 시 탑승(주문) 자체가 안 됩니다.

2. Audit Trail (감사 추적)

이게 뭔가요? 시스템에서 일어나는 **모든 행위**를 수정 불가능하게 기록합니다.

- 누가(Who), 언제(When), 무엇을(What), 왜(Why) 했는지
- 로그는 변경/삭제 불가 (Immutable)
- 금감원 검사관이 와서 "2024년 3월 15일 오전 10시 23분에 누가 삼성전자를 왜 팔았는지 보여주세요"라고 하면, 즉시 제출 가능해야 함

비유: 비행기 블랙박스. 사고가 나든 안 나든 항상 녹화하고 있음.

3. Regulatory Reporting (규제 보고)

금감원에 매일 제출해야 하는 보고서:

- **일일 VaR 보고서:** 전일 VaR vs 실제 손익 비교
- **대량 보유 보고:** 특정 종목 5% 이상 보유 시 공시
- **단기매매차익 보고:** 내부자의 6개월 이내 반대매매
- **자기자본비율:** 바젤 III 기준 충족 여부

Part 5: Must-Have — Stress Testing

왜 필수인가?

백테스트는 "과거에 일어난 일"을 보여줍니다. 스트레스 테스트는 **"아직 일어나지 않은 최악의 상황"**을 보여줍니다. 2008년 금융위기 때, 대부분의 은행은 "우리 VaR 괜찮아요"라고 했습니다. 하지만 VaR가 예측하지 못한 극단적 상황이 왔고, 리먼 브라더스가 파산했습니다.

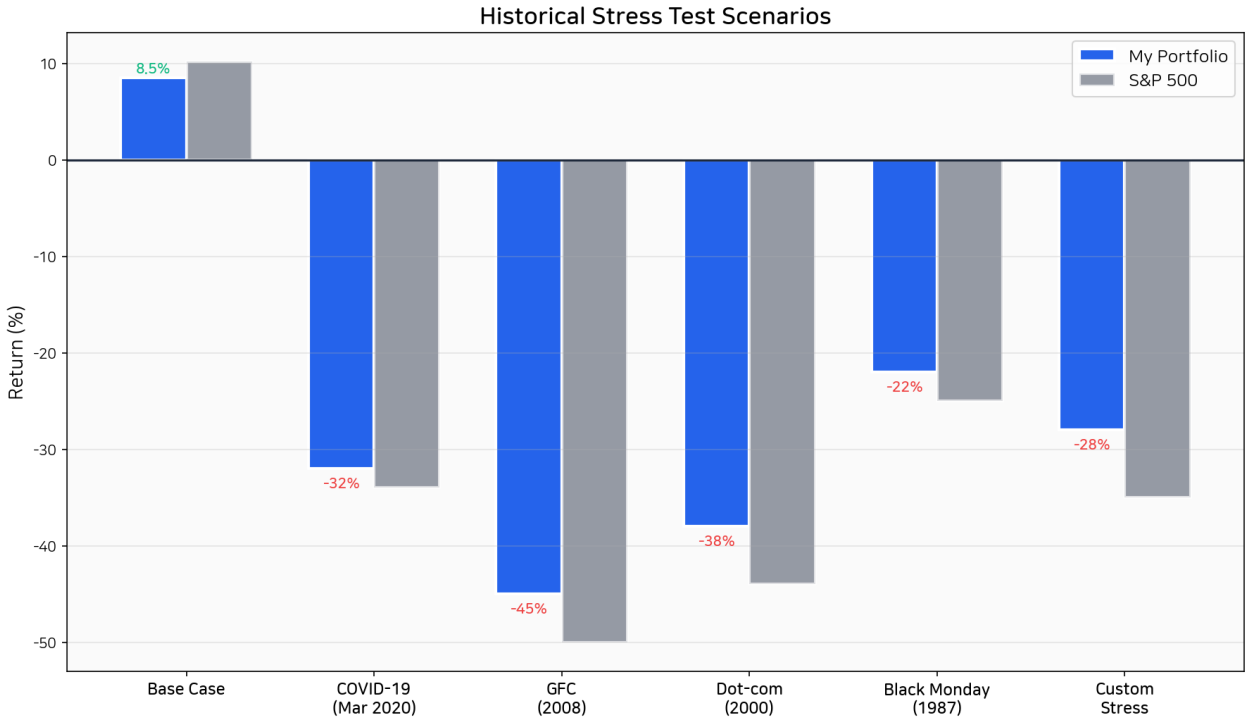


Figure 4. 역사적 위기 시나리오별 포트폴리오 영향 시뮬레이션

구현할 시나리오

시나리오	내용	적용 방법
Historical Replay	과거 위기를 그대로 재현	2008년 GFC, 2020년 COVID, 1997년 IMF 당시의 일별 수익률을 현재 포트폴리오에 적용
Hypothetical	가상의 시나리오	"한국 금리 2% 인상 + 원달러 1,500원 + KOSPI -20%"를 동시에 적용
Sensitivity	특정 변수만 변화	"금리가 1bp 오르면 채권 포트폴리오는 얼마나 손실?" (DV01 분석)

Reverse Stress	결과에서 역으로 추적	"우리가 파산하려면 어떤 시나리오가 필요한가?" → 가장 취약한 지점 발견

증권사에서 어떻게 쓰이나?

바젤 III 규제에 의해 모든 은행/증권사는 **분기별 스트레스 테스트 결과를 금감원에 보고**해야 합니다. 미국 Fed는 매년 대형 은행에 CCAR(Comprehensive Capital Analysis and Review)를 시행합니다. 스트레스 테스트를 통과하지 못하면 배당 지급이 금지됩니다.

Part 6: Must-Have — Multi-tenancy & Audit

멀티테넌시란?

여러 팀/부서가 **같은 시스템을 쓰되, 서로의 데이터를 볼 수 없게** 하는 구조입니다.

비유: 아파트. 같은 건물에 살지만 각자 집은 분리되어 있고, 남의 집 열쇠로 내 집 문을 열 수 없습니다.

증권사의 부서별 접근 권한

역할	볼 수 있는 것	할 수 있는 것
트레이더	자기 포트폴리오, 시장 데이터	주문 입력, 백테스트 실행
포트폴리오 매니저	팀 전체 포트폴리오	전략 설정, 리밸런싱 실행
리스크 매니저	전사 포지션, 리스크 지표	한도 설정, 경고 발령, 강제 청산
컴플라이언스	감사 로그, 거래 이력	보고서 생성, 의심 거래 조사
경영진	전사 대시보드, 핵심 KPI	조회만 가능

Part 7: Should-Have — Transaction Cost Analysis

TCA가 뭔가요?

주문을 넣는 순간의 가격과 실제 체결된 가격의 **차이를 분석**하는 것입니다.

비유: 온라인 쇼핑에서 "결제" 버튼을 누를 때 10,000원이었는데, 실제 카드에서는 10,150원이 빠졌다면? 그 150원이 어디로 갔는지 분석하는 겁니다.

비용 분해

$$\text{Total Cost} = \text{Commission} + \text{Spread} + \text{Market Impact} + \text{Timing Cost} + \text{Opportunity Cost}$$

비용 항목	설명	비유
Commission	증권사 수수료	택시 기본요금
Spread	매수호가와 매도호가의 차이	환전소 살 때/팔 때 가격 차이
Market Impact	내 대량 주문이 가격을 밀어올림	한꺼번에 100인분 주문하면 식당 가격이 오르는 것
Timing Cost	의사결정~실행 사이 가격 변동	"사야겠다" 생각하고 앱 켜는 동안 가격이 바뀜
Opportunity Cost	주문이 체결 안 되어 놓친 수익	매진된 콘서트 티켓. 살 수 없어서 잃은 기회

왜 중요한가?

기관투자자의 거래 비용은 연간 수익의 **1-3%**를 차지합니다. 연 10% 수익을 내는 펀드의 TCA가 2%라면, 실제 수익은 8%입니다. TCA를 0.5%만 줄여도 엄청난 차이가 납니다. **대형 자산운용사는 TCA 분석만 전담하는 팀**이 있습니다.

Part 8: Should-Have — Performance Attribution

성과 귀인 분석이란?

포트폴리오 수익을 **"어디서 벌었고 어디서 잃었는지"** 분해하는 것입니다. 현재 우리 Fama-French 5팩터 분석보다 훨씬 실용적인 버전입니다.

Brinson Attribution (가장 많이 쓰이는 방법)

Total Active Return = Allocation Effect + Selection Effect + Interaction Effect

구성요소	무엇을 측정?	비유
Allocation Effect (자산배분 효과)	"어떤 섹터에 돈을 더 넣었는가"의 효과	시험에서 어떤 과목에 시간을 더 할당했는지
Selection Effect (종목선택 효과)	"그 섹터 안에서 어떤 종목을 골랐는가"의 효과	같은 과목 안에서 어떤 문제를 먼저 풀었는지
Interaction Effect (상호작용)	배분+선택의 시너지/역효과	좋은 과목에서 좋은 문제를 골라 풀었는지

예시: "이번 분기 벤치마크 대비 +2%를 기록했는데, IT 섹터 오버웨이트(배분효과 +1.5%)와 삼성전자 대신 SK하이닉스를 선택한 것(종목선택 +0.7%)이 기여했고, 에너지 섹터 언더웨이트(-0.2%)가 감점 요인이었다."

증권사에서 어떻게 쓰이나?

펀드매니저의 **성과 평가에 직접 사용**됩니다. "당신의 초과수익 3%는 종목 선택이 아니라 시장 타이밍 때문이었습니다" 같은 분석이 가능합니다. 이 결과가 보너스 책정에 반영됩니다.

Part 9: Architecture Blueprint

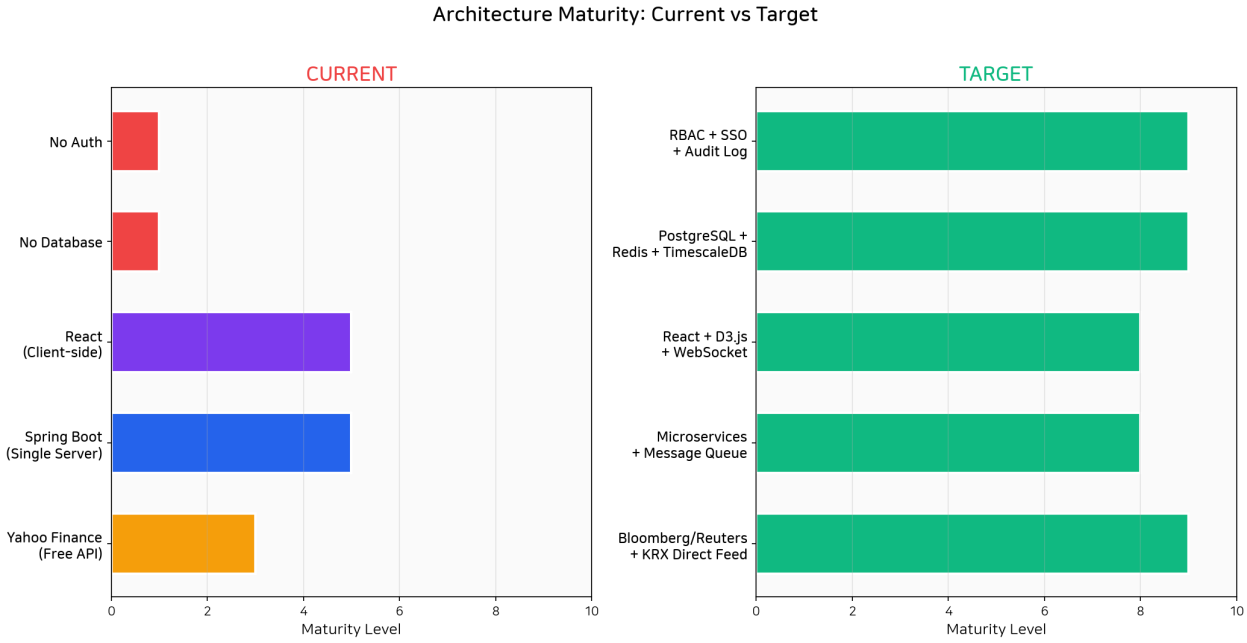


Figure 5. 현재 아키텍처(왼쪽) vs 목표 아키텍처(오른쪽) 성숙도 비교

목표 아키텍처 상세

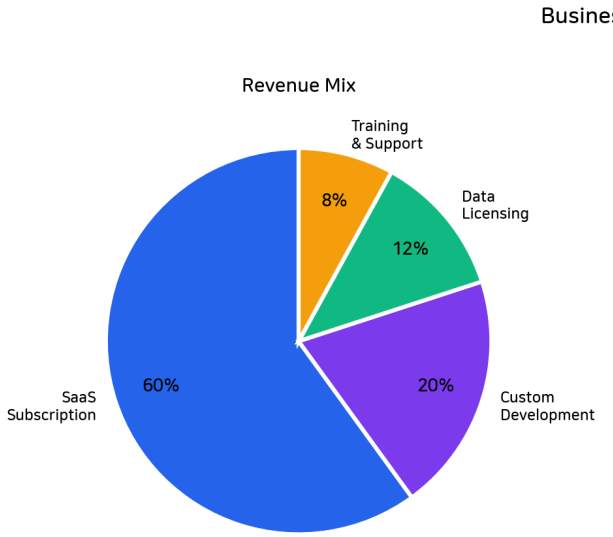
Layer	현재	목표	변경 이유
Frontend	React + Recharts	React + D3.js + AG Grid + WebSocket	실시간 차트, 대용량 테이블, 드래그 앤드롭 필요
API Gateway	없음	Kong / Spring Cloud Gateway	인증, 레이트 리미팅, API 버전 관리
Backend	Spring Boot (Monolith)	Microservices (Spring Boot + gRPC)	Risk Engine, Strategy Engine, Data Service 분리. 독립 배포/스케일링
Message Queue	없음	Apache Kafka	실시간 이벤트 스트리밍, 서비스 간 비동기 통신
Database	없음 (In-memory only)	PostgreSQL + TimescaleDB + Redis	RDBMS(포트폴리오, 사용자) + 시계열(가격) + 캐시(실시간)

Monitoring	없음	Prometheus + Grafana + ELK	시스템 모니터링, 로그 수집, 알림
Infra	로컬 실행	Kubernetes (on-premise 또는 private cloud)	증권사는 퍼블릭 클라우드 사용 제한. 자체 DC 또는 금융 클라우드

중요: 왜 퍼블릭 클라우드가 아닌가?

한국 금융규제상 고객 데이터와 거래 데이터는 **국내 데이터센터**에 보관해야 합니다. AWS Seoul Region은 일부 허용되지만, 대부분의 대형 증권사는 자체 DC 또는 금융권 전용 클라우드(NHN클라우드 금융존, KT금융클라우드)를 사용합니다.

Part 10: Business Model & Pricing



Pricing Tiers

Tier	Monthly	Features
Basic	$2,000	Backtest + Portfolio + Risk Metrics
Pro	$5,000	+ Real-time + Alerts + Tax
Enterprise	$15,000+	+ Custom Strategy + API + Support

Figure 6. 매출 구조 및 가격 티어

경쟁사 가격 벤치마크

솔루션	월 가격	대상
Bloomberg Terminal	$2,000/사용자	트레이더, 리서치
Aladdin (BlackRock)	$50,000+ (기관)	자산운용사
FactSet	$1,200/사용자	리서치, 포트폴리오
Portfolio Visualizer	$30~50/개인	개인투자자
Our Target	**$2,000~15,000**	**중소형 증권사/자산운용사**

포지셔닝 전략

Bloomberg/Aladdin은 너무 비싸고, Portfolio Visualizer는 너무 약합니다. 우리는 그 **사이**를 노립니다. "Bloomberg의 80% 기능을 20% 가격에" — 이것이 중소형 금융기관이 원하는 것입니다.

예상 수익 모델

항목	Year 1	Year 2	Year 3
고객 수 (기관)	3	10	25
평균 월 매출/고객	$5,000	$7,000	$8,000
연 매출	$180,000	$840,000	$2,400,000
커스텀 개발 매출	$100,000	$200,000	$300,000
총 매출	**$280,000**	**$1,040,000**	**$2,700,000**

Part 11: Implementation Roadmap

3단계 진화 전략

Stage 1: Foundation (3-6개월)

목표: "개인 투자 도구"에서 "프로토타입 B2B 솔루션"으로

- PostgreSQL + Redis 도입 (데이터 영속성)
- Spring Security + JWT 인증/권한 체계
- Audit Trail 기본 구현 (모든 API 호출 로깅)
- WebSocket 실시간 가격 업데이트 (KRX 데이터)
- Docker + Docker Compose 배포 체계

이 단계 완료 시: 핀테크 스타트업이나 소형 자산운용사에 데모 가능

Stage 2: Professional (6-12개월)

목표: "데모"에서 "파일럿 고객 확보"로

- Kafka 기반 실시간 파이프라인
- Pre-trade risk check 엔진
- Stress Testing 모듈 (Historical + Hypothetical)
- Brinson Performance Attribution
- RBAC (역할별 접근 제어) 완성
- Kubernetes 배포 + 모니터링 (Prometheus/Grafana)

이 단계 완료 시: 중소형 증권사 파일럿 프로젝트 수주 가능

Stage 3: Enterprise (12-24개월)

목표: "파일럿"에서 "정식 납품"으로

- Bloomberg/Refinitiv 데이터 피드 연동
- OMS/EMS FIX 프로토콜 연동

- TCA (Transaction Cost Analysis) 엔진
- 금감원 규제 보고서 자동 생성
- 고가용성 (Active-Active, 99.9% SLA)
- 보안 감사 (금융보안원 인증)

이 단계 완료 시: 대형 증권사 RFP 참여 가능

팀 구성 (최소 인력)

역할	인원	필요 역량
Backend / Infra	2-3	Spring Boot, Kafka, K8s, 금융 도메인
Frontend	1-2	React, D3.js, WebSocket, UX
Quant / Domain	1	금융공학, 리스크 모델링, 규제 지식
QA / DevOps	1	테스트 자동화, CI/CD, 보안
합계	**5-7**	

최종 정리

현재 상태: 증권사 납품 불가 (9개 영역 중 7개 F)

Stage 2 완료 시: 중소형 증권사 파일럿 가능
Stage 3 완료 시: 대형 증권사 RFP 참여 가능

한 줄 결론

지금 우리가 가진 건 "레시피"입니다. 좋은 레시피지만, 레스토랑을 열려면 주방 설비, 위생 인허가, 직원 교육, 식자재 공급망이 필요합니다. 레시피의 맛은 이미 검증됐으니, 이제 레스토랑을 짓는 일만 남았습니다.